

From Governance Artifact to Executable Guardrail: Characterizing Requirement-Enforcement Boundaries with Frozen Guardrails

Nitin Garg
Independent Researcher
Bothell, WA, USA
ORCID 0009-0007-5497-1311 |
nitingar@gmail.com

Harpreet Singh Siddhu
Independent Researcher
Warrington, PA, USA
ORCID 0009-0008-5397-7495 |
hpsiddhu@gmail.com

Jay Jignesh Dave
Independent Researcher
Raleigh, North Carolina, USA
ORCID 0009-0008-1917-0372 |
davejay1999@gmail.com

Abstract—*Motivated by AI-assisted software development, this paper studies how governed natural-language requirements can be transformed into executable guardrails without hiding the boundary to policy and human judgment. The controlled evaluation is performed at the requirement-and-evidence layer rather than by compiling or executing AI-generated application code. We construct a synthetic, model-assisted benchmark of 145 enterprise requirements across eight services and classify them with a frozen fully executable (FE), partially executable (PE), or non-executable (NE) codebook. The benchmark contains 97 FE (66.9%), 32 PE (22.1%), and 16 NE (11.0%) requirements. A structurally blinded 30-requirement validation freezes guardrails before a structurally blinded mutation pass creates 90 seeded challenge variants from requirement text without access to predicate internals. The protocol-wide challenge response is 73/90 (81.1%; Wilson 95% CI 71.8–87.9), with 53/60 FE (88.3%) and 20/30 PE (66.7%). All 73 non-policy/context variants are detected, whereas 17 policy/context probes produce no deterministic verdict; this is interpreted as an enforcement-boundary result rather than production recall. Across 90 varied valid controls, no false positive is observed (95% Wilson upper bound 4.1%), and requirement identity is preserved in 30/30 evidence chains. A 20-item public-source applicability sample from IETF, OpenAPI, and OWASP yields 12 FE and 8 PE classifications. The findings support a three-lane governance architecture: deterministic gates for FE requirements, executable evidence plus unresolved-context escalation for PE requirements, and explicit semantic review for NE requirements.*

Index Terms— executable specifications, requirements traceability, runtime verification, software governance, guardrails, AI-generated code, software assurance.

I. INTRODUCTION

AI coding assistants increasingly participate in implementation, yet functional plausibility is not evidence that generated behavior conforms to approved intent. Corso et al. found that code-assistant effectiveness degrades when tasks depend on context beyond a single class [10], and Fu et al. found security weaknesses in roughly a quarter to a third of analyzed AI-generated Python and JavaScript snippets from public projects [11]. These results motivate an assurance layer that is independent of the generator: generated code should be evaluated against stable, approved requirements rather than judged only by whether it compiles or passes local tests. In this study, AI-assisted development is the motivating application context; the experiment itself evaluates the requirement-to-guardrail and evidence layer and does not execute AI-generated application code.

Requirements engineering already treats traceability as a lifecycle concern. ISO/IEC/IEEE 29148 defines requirements management around identifying, documenting, maintaining, tracing, and tracking requirements, and links requirements to architecture, implementation elements, and verification entities [6]. NIST's Secure Software Development Framework similarly recommends executable-code testing to verify compliance with security requirements and emphasizes documentation, repeatability, and traceability [7]. The gap addressed here is operational: how can an approved natural-language requirement become an executable control while retaining its governance identity and its unresolved semantic portion?

Our prior GAISS study treated specifications as governance artifacts for human–AI review [1]. The

present work moves from passive review anchors to executable enforcement. The novelty is not runtime verification itself; Design by Contract, verification-aware programming, and runtime-monitoring research already demonstrate the utility of executable properties [2]–[5]. The contribution is a governance lineage that begins with an approved requirement and preserves its identity through enforceability classification, invariant generation, guardrail execution, evidence, and escalation.

The central hypothesis is deliberately bounded: not every requirement should be forced into a predicate. Instead, a useful governance system should distinguish the portion that can be checked deterministically from the portion that still needs contextual or human judgment. This leads to four research questions: RQ1—what fraction and kinds of requirements admit full or partial deterministic enforcement under a frozen codebook? RQ2—after guardrails are frozen, how do they respond to seeded challenge mutations generated without access to predicate internals, and how often do they reject valid controls? RQ3—where do deterministic controls stop, particularly for policy/context semantics? RQ4—can requirement identity remain traceable through evidence, and does the taxonomy remain applicable to a small sample of public requirements?

We contribute (1) an FE/PE/NE enforcement taxonomy; (2) a requirement-identity-preserving guardrail architecture; (3) a reproducible 145-requirement controlled benchmark; (4) a structurally blinded 30-requirement challenge protocol with frozen guardrails, 90 seeded challenges, and 90 valid controls; (5) traceability evidence across the full requirement-to-evidence chain; and (6) a 20-item public-source applicability check. All

empirical claims are scoped to synthetic fixtures and seeded challenges; no production recall, deployment-effectiveness, model-superiority, or repair-success claim is made.

II. RELATED WORK AND POSITIONING

Design by Contract expresses component obligations as preconditions, postconditions, and invariants [2]. Dafny makes specifications executable within a verification-oriented programming model [5]. Runtime verification observes executions against formal properties, while Monitoring-Oriented Programming can synthesize monitors from user-specified properties and associate actions with violations [3], [4]. These approaches provide mature enforcement mechanisms once a formal property exists. Our concern is the upstream governance transformation: deciding whether an approved requirement is fully executable, partially executable, or intrinsically semantic, then preserving the requirement identifier and unresolved context through the enforcement lifecycle.

The distinction also complements secure-development guidance. NIST SSDF calls for testing executable code to identify vulnerabilities and verify compliance with security requirements, noting the traceability and repeatability benefits of automated methods [7]. NIST IR 8397 recommends minimum developer-verification techniques such as threat modeling, automated testing, static analysis, and review [8]. NIST AI 600-1 extends risk-management guidance to generative AI and emphasizes lifecycle measurement and documentation [9]. Our framework can host these mechanisms beneath a stable governance mapping rather than replacing them.

AI-code studies reinforce why a stable verification layer matters. The ICPC 2024 study by Corso et al. showed that code-assistant accuracy drops sharply as external context grows [10]. Fu et al.'s TOSEM study of 733 snippets found security weaknesses across 43 CWE categories and reported that generated-code security issues remain diverse even when repair assistance is available [11]. Functional benchmarks such as HumanEval measure whether generated programs pass hidden tests [12], but do not establish that the generated behavior satisfies organization-specific policy, workflow, audit, or accountability requirements.

TABLE I. POSITIONING AGAINST ADJACENT ASSURANCE APPROACHES

Approach	Gov. req.?	Exec. check	Unresolved semantics	Req.-linked evidence
Design by Contract [2]	No	Yes	No	Indirect
Runtime verification [3], [4]	No	Yes	No	Property-linked
SSDF guidance [7], [8]	Policy/process	Mechanism-dependent	Process-level	Recommended
GAISS prior work [1]	Yes	Review-time	Yes	Yes
This work	Yes	FE/PE	Yes, explicit	Yes, stable ID

III. ARCHITECTURE AND ENFORCEMENT MODEL

Every approved requirement first receives a stable identifier and an enforceability decision. FE requirements are compiled into complete deterministic predicates. PE requirements produce an executable sub-predicate plus an explicit unresolved-context marker. NE requirements are not converted into brittle approximations; they are routed directly to semantic or human review. This boundary is a first-class artifact rather than an implementation caveat.

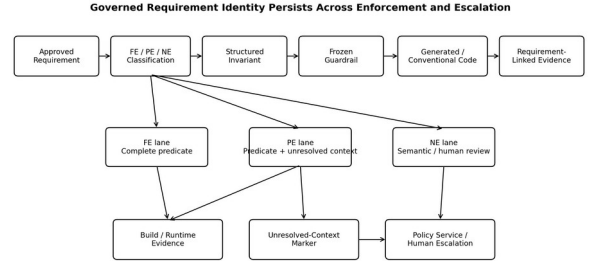


Fig. 1. Governance-to-guardrail architecture with explicit FE, PE, and NE enforcement lanes.

A structured invariant contains `requirement_id`, service, category, predicate type, required evidence, executable portion, unresolved portion, severity, and failure action. Supported predicate families in the prototype include numeric comparison, required fields, cardinality/idempotency, state transition, authorization, cross-record equality, audit-event presence, and temporal thresholds. The runtime/build evidence record repeats the requirement identifier. For a PE requirement, a passing predicate means only that its executable portion passed; it does not imply that the whole governed requirement is compliant.

This design operationalizes bidirectional traceability in a way aligned with requirements-engineering practice [6]. The requirement can be traced downward to a guardrail and evidence record, while a failure can be traced upward to the approved requirement that motivated the control. The same lineage also supports bounded repair or regeneration, but repair effectiveness is outside this study.

IV. METHOD

A. Constructed Benchmark and Frozen Codebook

The controlled benchmark contains 145 author/model-assisted requirements across payment, refund, insurance-premium, claims-payout, account-transfer, order-processing, subscription-billing, and approval-workflow services. The benchmark is synthetic by design so the study can be reproduced without proprietary requirements. Nine categories cover input/value, schema/data contract, idempotency/cardinality, state-machine, authorization/security, cross-component consistency, observability/audit, temporal/eventual consistency, and semantic/human judgment.

The codebook is frozen before execution. FE requires a complete deterministic pass/fail predicate, programmatically obtainable evidence, and no semantic/human judgment. PE requires a meaningful deterministic portion but retains external context, delayed

state, policy interpretation, semantic interpretation, or human judgment for complete compliance. NE applies when the core compliance condition cannot reliably be reduced to a deterministic predicate. The tie-break is conservative: if meaningful machine-checkable structure exists but full compliance remains unresolved, the requirement is PE rather than NE.

As a blinded human consistency check, a second human reviewer who was not involved in benchmark construction, taxonomy design, or reference labeling independently applied the frozen FE/PE/NE codebook to 40 held-out requirements spanning all eight services and nine categories. The reviewer saw the requirement text, service, and category, but not the reference classification, and agreed with all 40 reference labels (20 FE, 12 PE, 8 NE; 100% raw agreement; Cohen's kappa = 1.00).

The second reviewer was not an author of this paper and had no role in its preparation.

Because the constructed benchmark uses deliberately regular category templates and the held-out sample was not designed to maximize ambiguous FE/PE boundary cases, this result is interpreted as confirmation of classification consistency within the controlled benchmark rather than evidence of general inter-rater reliability on naturally occurring requirements.

B. Frozen-Guardrail Challenge Protocol

Thirty FE/PE requirements were selected using a predeclared stratified list spanning all eight services and multiple executable categories; the set contains 20 FE and 10 PE requirements. The 30 frozen guardrails were produced by a deterministic rule-based compiler: the frozen requirement category and FE/PE classification select one of eight predefined Python predicate templates and its required evidence schema. No LLM or human authored or edited an individual challenge guardrail, and no manual fixes were applied. The resulting predicate definitions were serialized, hashed, and frozen before any challenge mutations were generated. The challenge protocol does not compile or execute AI-generated application or service code; it evaluates these frozen Python guardrail predicates against structured evidence records that represent requirement-relevant system state.

The mutation-generation pass received only requirement ID, service, category, and blinded natural-language text. Its input schema rejected classification and predicate fields. Three variants were generated for each requirement (90 total). A challenge row is one synthetic structured evidence record associated with a blinded requirement and generated from its requirement category using a predefined mutation family to represent a seeded violation or policy/context probe; it is then evaluated once against the corresponding frozen guardrail. Mutation families included numeric boundary, missing/extra structure, duplicate/replay, invalid state transition, authorization context, cross-record inconsistency, missing audit evidence, delayed/late state, multi-field interaction, and policy/context probes. All mutations were frozen before guardrail execution. This design removes direct predicate visibility, but both sides still share the

same high-level category taxonomy; Section VII treats this as residual design coupling.

Each challenge result records the frozen mutation identifier, requirement ID, mutation class, expected challenge flag, guardrail result, detection indicator, evidence, and run ID. The primary protocol-wide metric is Seeded Challenge Detection Rate = detected challenge rows / all seeded challenge rows. Because some policy/context probes intentionally alter only unresolved semantic context, we also report the distribution by mutation class rather than treating a single aggregate as a production-recall estimate.

C. Valid Controls, Traceability, and Statistics

Three valid controls were generated per challenge requirement (90 total). For PE requirements, the third control explicitly exercises a context-dependent exception while satisfying the executable condition. The Seeded Valid-Case False Positive Rate is failed valid controls divided by 90. Requirement traceability passes only when the same requirement ID is present in the requirement, frozen classification, guardrail definition, mutation link, and result evidence.

The analysis is descriptive. We report proportions and Wilson 95% confidence intervals for binomial challenge outcomes. No p-values or superiority tests are used because the challenge suite is synthetic and not a random sample from a defect population. Zero observed false positives are therefore reported with an upper confidence bound rather than as proof of a zero operational rate.

D. Public-Source Applicability Sample

To check whether the codebook can be applied outside the constructed templates, 20 short public requirements were sampled from IETF RFC 9110 (HTTP Semantics), RFC 7519 (JWT), RFC 6749 (OAuth 2.0), OpenAPI 3.1.1, and the OWASP Authentication Cheat Sheet [13]–[17]. Each record preserves source URL, section, and faithful requirement text. The sample is a heterogeneous convenience set, not a representative corpus and not an estimate of real-world FE/PE/NE prevalence.

V. RESULTS

A. Enforceability Coverage (RQ1)

The constructed benchmark contains 97 FE (66.9%), 32 PE (22.1%), and 16 NE (11.0%) requirements. Therefore 129/145 (89.0%) admit at least partial machine-checkable enforcement under the frozen codebook. The result should be read as controlled coverage: input/value, schema, idempotency/cardinality, and state-machine categories were generated as FE; authorization, cross-component consistency, observability, and temporal categories contain equal FE/PE coverage; semantic/human-judgment requirements were generated as NE.

TABLE II. CONTROLLED BENCHMARK ENFORCEABILITY COVERAGE

Class	Count	Share	Operational treatment
FE	97	66.9%	Complete

			deterministic predicate
PE	32	22.1%	Predicate + unresolved context
NE	16	11.0%	Semantic/policy/human review
FE+PE	129	89.0%	At least partial machine-checkability

This regularity is a limitation, not hidden evidence of natural prevalence. The 89.0% figure should not be generalized as '89% of software requirements are automatable.' Its value is to exercise the proposed enforcement lanes systematically and to support the downstream challenge experiment.

B. Guardrail Generation and Seeded Challenges (RQ2)

All 30 selected FE/PE guardrails were generated and compiled successfully without manual fixes. Across 90 seeded challenge rows, 73 produced deterministic failures, yielding a protocol-wide response of 81.1% (Wilson 95% CI 71.8–87.9). The FE subset produced 53/60 detections (88.3%, 95% CI 77.8–94.2), while the PE subset produced 20/30 (66.7%, 95% CI 48.8–80.8). These values characterize this frozen synthetic challenge suite, not production recall.

TABLE III. FINAL CONTROLLED CHALLENGE RESULTS

Measure	Outcome	Interpretation
Generated/compiled guardrails	30/30	No manual fixes
Protocol-wide challenges	73/90 (81.1%)	All frozen challenge rows
FE challenges	53/60 (88.3%)	Includes 7 context-only probes
PE challenges	20/30 (66.7%)	Includes 10 context-only probes
Valid controls	0/90 rejected	95% Wilson upper bound 4.1%
Traceability	30/30 complete	Stable ID across artifact chain

C. Boundary-Probe Analysis (RQ3)

The mutation-class breakdown is the paper's most informative result. Every one of the 73 non-policy/context variants was detected, whereas all 17 policy/context probes produced no deterministic failure. Ten policy/context probes were attached to PE requirements, where that behavior is expected: the executable predicate can pass while the unresolved policy portion still requires escalation. Seven probes were attached to FE requirements; inspection of the raw evidence shows that those variants added an unresolved-policy-context marker but did not change the observable fields used by the FE predicate.

Accordingly, the 17 rows should not be interpreted as conventional false negatives of a ground-truth machine predicate. They are boundary probes: they ask whether a deterministic guardrail can decide a policy/context statement that is not encoded in its evidence relation. We retain them in the protocol-wide denominator to avoid

post-hoc score inflation, but separately expose their mutation class. A scope-aware reading is therefore: all 73 taxonomy-aligned machine-grounded challenge cases exercised their intended deterministic failure path, while 0/17 policy/context probes produced a deterministic failure. This verifies implementation behavior within the controlled framework rather than estimating independent detection robustness, because the mutation and predicate frameworks share category-level design assumptions.

D. Valid Controls and Traceability (RQ2/RQ4)

No valid control was rejected (0/90). The controls include normal variations and, for PE requirements, a context-dependent exception case with the executable condition satisfied. Zero observed failures does not establish a zero operational false-positive rate; the Wilson 95% upper bound is 4.1%. The traceability chain was complete for 30/30 requirements: requirement text → frozen classification → frozen guardrail → mutation link → result evidence retained the same requirement identifier.

E. Public-Source Applicability Check (RQ4)

TABLE IV. PUBLIC-SOURCE APPLICABILITY SAMPLE

Source	FE	PE	NE	Total
IETF RFC 6749	4	0	0	4
IETF RFC 7519	3	2	0	5
IETF RFC 9110	3	5	0	8
OWASP Auth. Cheat Sheet	0	1	0	1
OpenAPI Spec. 3.1.1	2	0	0	2
Total	12	8	0	20

The 20 public requirements yield 12 FE (60%) and 8 PE (40%) classifications, with no NE items. Examples include deterministic JWT expiration and duplicate-claim rules, and partially executable HTTP or security statements whose complete interpretation depends on context. The absence of NE items is unsurprising because normative standards language preferentially states explicit MUST/SHOULD obligations; no representativeness claim is made.

F. Cross-Service Robustness and Failure Semantics

Aggregate FE/PE rates can conceal whether one service dominates the result. Table V therefore reports the frozen challenge outcomes by service. All eight services are represented, with protocol-wide response rates from 75.0% to 91.7%. Payment has the highest response (11/12), while insurance-premium, order-processing, and refund each produce 9/12. The narrow range is expected because the service fixtures share an enforcement framework, but it confirms that the final score is not produced by a single favorable domain.

The service-level view does not remove the more important class-level asymmetry. PE requirements lose deterministic coverage when the challenge targets the unresolved portion of the requirement. To distinguish a guardrail failure from a scope mismatch, we separate three concepts: (i) a machine-grounded violation changes evidence consumed by the frozen predicate; (ii) a boundary probe changes only policy/context state that the predicate was never defined to decide; and (iii) a valid control preserves both the executable condition and any modeled exception. This distinction is essential because collapsing (i) and (ii) into one binary 'recall' metric would imply that a deterministic checker is responsible for semantic facts that were explicitly excluded by the codebook.

In the raw challenge package, all 73 machine-grounded variants are rejected and all 17 policy/context probes pass the deterministic predicate. Ten of the 17 are attached to PE requirements and directly exercise their unresolved portion. Seven are attached to FE requirements even though their mutation evidence does not alter the FE predicate inputs. We therefore retain the original 73/90 protocol-wide score for transparency, while interpreting the 17 context probes as evidence about the automation boundary rather than conventional false negatives.

G. Representative Enforcement Cases

The FE case illustrates why idempotency/cardinality rules are good guardrail candidates: a stable business-intent key and effect count provide a complete predicate. The PE case shows the opposite failure mode of naive automation: comparing two values is easy, but deciding whether a documented exception is authorized requires policy context. The NE case makes the semantic boundary explicit. Checking for a non-empty message or keywords can provide supporting evidence, but cannot establish that the message is clear or that the recommended action is appropriate for a specific user and business context.

This separation avoids treating an incomplete proxy as full compliance. A PE guardrail emits machine evidence plus an unresolved-context marker, which can route the remaining decision to a policy engine, reviewer, or accountable human while preserving the requirement identifier.

VI. DISCUSSION

A. What the Findings Support

The empirical story is not that natural-language governance has been fully automated. Rather, the results support a routing architecture. Machine-expressible clauses can be compiled into repeatable gates; partially executable clauses can expose measurable sub-conditions while preserving unresolved context; semantic clauses should remain explicit review obligations. In the challenge experiment, the separation appears operationally meaningful: machine-grounded variants were consistently rejected, whereas policy/context probes received no deterministic verdict.

B. Novelty Relative to Runtime Verification

The contribution is best understood as an orchestration layer around established assurance mechanisms. Runtime verification asks whether a trace satisfies a property; Design by Contract asks whether a component satisfies explicit obligations [2]–[4]. Our framework asks an upstream question first: what portion of an approved governance requirement is safe to turn into such a property? It then records the unresolved remainder rather than erasing it. Three properties distinguish the approach: governance-originated identity, explicit FE/PE/NE triage, and first-class preservation of unresolved PE semantics.

C. Design Implications for AI-Assisted Delivery

An executable-governance pipeline should make the classification decision visible to developers rather than hiding it in tooling. For FE requirements, the build or runtime outcome can be binary because the evidence relation is complete. For PE requirements, the evidence record should present two fields: `executable_status` and `unresolved_context_status`. A passing `executable_status` must never be rendered as 'requirement compliant' while `unresolved_context_status` remains open. For NE requirements, the system should record the required reviewer or policy authority instead of synthesizing a weak proxy.

D. Comparative Interpretation Against Prior Work

This comparison sharpens the novelty claim: existing verification techniques generally assume the property to be checked is already known in executable form, and requirements standards emphasize traceability but do not prescribe an FE/PE/NE enforcement decision for AI-assisted implementation. Our contribution sits between those layers, preserving one identity across classification, enforcement, and escalation.

VII. THREATS TO VALIDITY

Construct validity is limited by the frozen FE/PE/NE codebook. Whether evidence is 'reasonably obtainable programmatically' can vary with architecture, telemetry, policy stores, and domain modeling. The blinded human consistency check achieved 100% agreement on the 40 held-out items, but the perfect score likely reflects in part the intentionally regular structure of the constructed benchmark and the inclusion of category metadata. Establishing broader inter-rater reliability would require multiple independent raters on naturally occurring requirements, ideally with category metadata withheld and with explicit oversampling of difficult FE/PE boundary cases.

Internal validity is improved by guardrail freezing, hashing, overwrite protection, mutation-schema gating, and freezing all mutations before execution. However, both the guardrail and mutation generators share the same broad requirement taxonomy, creating residual design coupling. The seven FE context-only probes further show that synthetic mutation generators can produce challenge rows whose semantic target

lies outside the predicate evidence. We therefore report mutation-class outcomes and preserve the original protocol-wide denominator rather than silently correcting the score. A fully decoupled evaluation would require a third-party-authored challenge set, or a separate model/process with no shared design history or access to the guardrail-generation workflow.

External validity is limited by author/model-constructed requirements, synthetic service fixtures, seeded rather than naturally occurring defects, and local execution. The 20 public requirements provide an applicability check but are a small convenience sample dominated by normative standards. The 0/90 valid-control outcome is not an operational false-positive estimate. The study does not evaluate real organizational policy engines, production traffic, model-specific generation quality, distributed failure modes, or repair success.

Conclusion validity is descriptive. Confidence intervals quantify uncertainty for the finite seeded suite but do not make the suite representative of production defects. No *p*-values, causal claims, or statistical-superiority claims are made.

VIII. REPRODUCIBILITY AND ARTIFACTS

The artifact package preserves the constructed benchmark and codebook, frozen challenge selections and guardrails, hashes and freeze evidence, blinded mutation requests and generated mutations, row-level challenge and valid-control outcomes, traceability rows, public-source provenance, executable scripts, tests, final metrics, integrity audits, and manifest files. Metrics are recomputable from stored CSV/JSON artifacts without invoking an LLM. This supports audit of every reported count and makes unsupported metrics—such as production recall, repair success, or model superiority—explicitly unavailable rather than implicitly inferred.

IX. CONCLUSION

Executable guardrails can move governed specifications from passive documentation toward repeatable evidence without pretending that every requirement is formalizable. In a controlled 145-requirement benchmark, 97 requirements were FE, 32 PE, and 16 NE under a frozen codebook. In a structurally blinded 30-requirement challenge set, frozen guardrails responded to 73/90 seeded challenge rows, rejected none of 90 valid controls, and preserved requirement identity across all 30 evidence chains. Mutation-class analysis showed a sharper boundary: all 73 taxonomy-aligned machine-grounded challenge cases exercised their intended deterministic failure path, while all 17 policy/context probes produced no deterministic failure. A small public-source sample showed the same FE/PE distinction can be applied outside the constructed templates.

The practical conclusion is therefore bounded but actionable: use deterministic enforcement where the requirement truly reduces to observable evidence, preserve unresolved semantics as first-class governance state, and route those semantics to policy or human

judgment. For AI-assisted development, this creates a stable accountability layer that can outlive any individual prompt, model, or generated implementation.

REFERENCES

- [1] N. Garg, R. Mohindra, and S. Maru, "Specifications as Governance Artifacts: Human-AI Collaboration and Oversight for AI-Generated Code," *GAISS*, 2026, accepted.
- [2] B. Meyer, "Applying 'Design by Contract'," *Computer*, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279.
- [3] F. Chen and G. Roşu, "MOP: An Efficient and Generic Runtime Verification Framework," in *Proc. OOPSLA*, 2007, pp. 569–588, doi: 10.1145/1297027.1297069.
- [4] C. Sánchez et al., "A Survey of Challenges for Runtime Verification from Advanced Application Domains (Beyond Software)," *Formal Methods in System Design*, vol. 54, pp. 279–335, 2019, doi: 10.1007/s10703-019-00337-w.
- [5] K. R. M. Leino, "Developing Verified Programs with Dafny," in *Proc. HILT*, 2012, pp. 9–10, doi: 10.1145/2402676.2402682.
- [6] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, 2018.
- [7] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1," *NIST SP 800-218*, 2022, doi: 10.6028/NIST.SP.800-218.
- [8] P. E. Black et al., "Guidelines on Minimum Standards for Developer Verification of Software," *NISTIR 8397*, 2021, doi: 10.6028/NIST.IR.8397.
- [9] C. Autio et al., "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," *NIST AI 600-1*, 2024, doi: 10.6028/NIST.AI.600-1.
- [10] V. Corso, L. Mariani, D. Micucci, and O. Riganelli, "Generating Java Methods: An Empirical Assessment of Four AI-Based Code Assistants," in *Proc. 32nd IEEE/ACM ICPC*, 2024, pp. 13–23, doi: 10.1145/3643916.3644402.
- [11] Y. Fu et al., "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 8, Art. 218, 2025, doi: 10.1145/3716848.
- [12] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.
- [13] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," *IETF RFC 9110*, June 2022.
- [14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015.
- [15] D. Hardt, "The OAuth 2.0 Authorization Framework," *IETF RFC 6749*, Oct. 2012.
- [16] OpenAPI Initiative, "OpenAPI Specification, Version 3.1.1," Oct. 2024.
- [17] OWASP Foundation, "Authentication Cheat Sheet," *OWASP Cheat Sheet Series*, accessed Aug. 2026.